



Project 1: Quest Build your Game: Number Guessing Game

Teresa Mendes Coelho

Ironhack Bootcamp Data Analytics June 2026



Game: Number Guessing Game

How to Play

Choose a difficulty level and try to guess the randomly generated number within the available attempts.

Game Rules

EASY

1-50
10 attempts

CLASSIC

1-100
10 attempts

HARD

1-100
8 attempts

- Feedback after every valid guess
- Invalid input does not consume an attempt
- Win by guessing correctly; lose when all attempts are used
- Play again and track session statistics



Game Logic

The gameplay follows a clear flow from input to feedback and game-state updates.

1 CHOOSE

Select difficulty
Set range + attempts

2 GENERATE

Create random
secret number

3 GUESS

Validate input
Store valid guess

4 FEEDBACK

Too high
Too low
Correct

5 UPDATE

Update attempts
and game state

6 END

Win / Lose
Play again



Data Structures & Game State

Two core data structures store and manage information during the game.

LIST — player guesses

`guesses = []`

Stores each valid guess made by the player.

Example
`[50, 25, 42]`

DICTIONARY — game state

`game_state = { ... }`

Tracks difficulty, secret number, attempts, guesses and win status.

The game state is updated after every valid guess.



Python Concepts Used in the Game

These concepts support the game logic, repeated actions, randomness, validation and code quality.

FLOW CONTROL

if / elif / else
Makes decisions based on game conditions.

LOOPS

for / while
Control repeated gameplay and validation.

MODULES & IMPORTS

random /
random.randint()
Generate a secret number for each game.

ERROR HANDLING

try / except
Handle invalid input without crashing the game.

DOCUMENTATION

Comments and docstrings
Explain the code and document functions.

Together, these concepts make the program structured, robust and easier to understand.



Python Functions & Methods Used

Python functions

print()
input()
int()
range()
random.randint()

Methods

.strip()
.lower()
.append()



Custom Functions & Responsibilities

Nine custom functions divide the program into clear, reusable responsibilities.

INPUT

`choose_difficulty()`
`get_player_guess()`

GAME LOGIC

`check_guess()`
`update_game_state()`

STATUS

`display_game_status()`
`check_game_over()`

GAME FLOW

`play_one_game()`
`play_again()`

SESSION

`number_guessing_game()`
Controls the complete session and calculates games played, wins and win rate.



Code Structure & Reusability

The main functions work together as a simple, reusable flow.

1 — SET UP

`choose_difficulty()`
Sets range and attempt limit.

2 — PLAY

`play_one_game()`
Runs one complete game.

3 — PROCESS

`get_player_guess()` →
`check_guess()` →
`update_game_state()`

4 — REPEAT

`play_again()`
Starts another game using the same logic.

Key idea: changing the difficulty changes only the range and attempt limit — the core game logic remains reusable.



Input Validation & Error Handling

Invalid input is rejected without crashing the game or consuming an attempt.

INPUT CASES

- Text instead of a number
- Empty input
- Number below minimum
- Number above maximum
- Invalid difficulty choice
- Invalid Play Again response

VALIDATION FLOW

try → convert input to int

ValueError → show error

Out of range → reject

Valid guess → process

Only valid guesses consume an attempt.

Result: more robust and predictable gameplay.



Testing & Edge Cases

Testing covered both normal gameplay and unexpected user input.

GAMEPLAY TESTS

- Correct guess
- Too high / too low
- Win on first attempt
- Win on final attempt
- Lose after all attempts
- Repeated guesses
- Multiple games

VALIDATION + SETTINGS

- Text / empty input
- Out-of-range numbers
- Invalid difficulty
- Invalid Play Again
- Easy / Classic / Hard
- Remaining attempts after every valid guess

The objective was to verify that the game behaves correctly — not only that it runs.



What I Learned

Building the game helped me connect Python concepts into one working application.

STRUCTURE

- Functions
- Lists
- Dictionaries
- Game state

PROBLEM SOLVING

- Input validation
- Error handling
- Testing edge cases

APPLICATION

- Difficulty levels
- Play again
- Session statistics

KEY TAKEAWAY

A simple idea became a structured, reusable and fully playable Python program.



Challenges & Problem Solving

The day only have 24h: I don't know what is to have proper night's sleep since I start this boot camp.

- **Technical challenges**

- Breaking the game into smaller functions
- Keeping the game state consistent
- Handling different input cases
- Debugging logic errors

- **What helped**

- Testing different scenarios
- Refactoring the code step by step
- Prioritising the most important issues first

- **Key takeaway:** The main lesson was learning to solve problems step by step instead of trying to fix everything at once.



Game Demo

- Try the game it self, lets play?

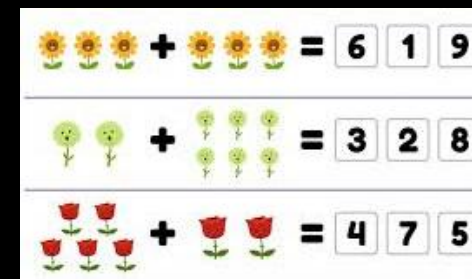
https://colab.research.google.com/drive/1TAS_oF8-H6mbzDOJYY2JANkww0eOS5j6#scrollTo=7QcXiZCpSWli

- The demo shows the complete gameplay flow, including difficulty selection, validation, feedback and replay.





```
void CSSdescramble(unsigned char *sec,unsigned char *key) {
    unsigned int t1,t2,t3,t4,t5,t6;
    unsigned char *end=sec+0x800;
    t1=key[0]^sec[0x54]|0x100;
    t2=key[1]^sec[0x55];
    t3=((!(unsigned int *) (key+2)))^((!(unsigned int *) (sec+0x56)));
    t4=t367;
    t3=t3*2+8-t4;
    sec+=0x80;
    t5=0;
    while(sec!=end) {
        t4=CSSt2[t2]^CSSt3[t1];
        t2=t1>>1;
        t1=((t1&1)<<8)^t4;
        t4=CSSt5[t4];
        t6=(((((t3>>3)^t3)>>1)^t3)>>8)^t3)>>5&0xff;
        t3=(t3<<8)|t6;
        t6=CSSt4[t6];
        t5+=t6+t4;
        *sec+=CSSt1[*sec]^(t5&0xff);
        t5>>=8;
    }
}
```



Concepboard

Any questions?



Thanks for much to be here for this apresentattion

